

LINE 飲食熱量精算師：Gemini 2.5 + GAS 完全建置手冊

摘要

本手冊旨在引導開發者從零開始建置一套整合 LINE Messaging API、Google Apps Script (GAS) 與 Gemini 2.5 旗艦版 AI 的多功能飲食管理機器人。使用者只需透過 LINE 傳送大肚便當或食物照片，系統即會透過 AI 自動辨識、計算熱量與三大營養素，並即時寫入 Google 試算表。本手冊包含完整的後台設定截圖對應指南、程式碼、以及防呆修正機制的配置。

內容

第一章：LINE Developers 後台完整設定指南

請瀏覽並登入 LINE Developers Console，依序執行以下設定：

1. 建立提供者 (Provider)

點擊頁面左側的「Create a new provider」。

輸入提供者名稱（例如：MyAIProject），然後點擊「Create」。

2. 建立 Messaging API 通道 (Channel)

在剛建立的 Provider 頁面中，點擊「Create a Messaging API channel」。

填寫必要資訊：

Channel name：輸入機器人的名字（例如：AI熱量精算師）。

Channel description：輸入簡短介紹（例如：透過 Gemini 2.5 幫您精算照片中的食物熱量）。

Category / Subcategory：選擇合適的分類（如健康、工具）。

Email address：填入您的聯絡信箱。

勾選同意服務條款，點擊「Create」。

3. 獲取並設定通道金鑰

獲取 Channel Access Token：

切換到「Messaging API」頁籤。

捲動到最下方，找到「Channel access token (long-lived)」。

點擊

「Issue」，系統會產生一串非常長的亂碼，請複製並妥善保存（此即為程式碼中的 LINE_ACCESS_TOKEN）。

關閉預設罐頭回應：

在同一個頁籤找到「LINE Official Account features」，點擊「Reply messages」旁邊的「Edit」。

系統會跳轉至 LINE 官方帳號管理後台。請將「自動回應訊息」關閉（Disabled），並將「進階設定 -> Webhook」開啟（Enabled）。

第二章：建立 Google 試算表與部署 GAS 程式

本步驟將建立資料庫，並將 AI 與 LINE 串接的橋樑（Webapp）建置起來。

1. 建立 Google 試算表 (Database)

瀏覽並新建一個全新的 Google 試算表。

不需要手動建立分頁或欄位標題，程式在第一次運行時會自動偵測，並以使用者的 LINE 顯示名稱自動建立專屬分頁（支援多人同時使用）。

複製網址列中的 試算表 ID（即網址中 /d/ 與 /edit 之間的那串英數字，此即為程式碼中的 SHEET_ID）。

2. 撰寫 Google Apps Script 程式碼

在試算表上方選單點擊「擴充功能」->「Apps Script」。

將預設的 myFunction 清空，並完整貼上以下最新的 Gemini 2.5 旗艦版完整程式碼：

```
// =====  
// 1. 系統核心設定區 (請在此替換您的專屬三大金鑰)  
// =====  
var LINE_ACCESS_TOKEN = '您的_LINE_CHANNEL_ACCESS_TOKEN';  
var GEMINI_API_KEY = '您的_GOOGLE_GEMINI_API_KEY';  
var SHEET_ID = '您的_GOOGLE_SPREADSHEET_ID';
```

```

// 核心大腦配置 (指向最新的 Gemini 2.5 旗艦型號)
var GEMINI_MODEL = "gemini-2.5-flash";

// =====
// 2. Webhook 主要入口門戶 (doPost)
// =====
function doPost(e) {
  var replyToken = "";
  try {
    var contents = JSON.parse(e.postData.contents);
    var event = contents.events[0];
    replyToken = event.replyToken;
    var userId = event.source.userId;
    var userName = getUserProfile(userId);
    // 【文字訊息流處理】
    if (event.message.type === 'text') {
      var userMsg = event.message.text.trim();

      if (userMsg.includes("本月") || userMsg.includes("月統計")) {
        replyMessage(replyToken, queryMonthlyNutrition(userName));
      }
      else if (userMsg.includes("今天") || userMsg.includes("統計") || userMsg.includes("今日")) {
        replyMessage(replyToken, queryTodayNutrition(userName));
      }
      else if (userMsg.includes("修正") || userMsg.includes("不對") || userMsg.includes("改")) {
        handleAdvancedCorrection(replyToken, userName, userMsg);
      }
      else {
        replyMessage(replyToken, "□ " + userName + " 您好！
        • 直接傳食物照片：Gemini 2.5 立即幫您計算熱量
        • 輸入「今天統計」：看今日飲食編號清單
        • 輸入「本月統計」：看本月總加總報表
        • 輸入「修正第 X 餐：[修正內容]」：可直接重算並覆蓋舊紀錄");
      }
    }
    // 【影像訊息流處理】
    if (event.message.type === 'image') {
      var imageBlob = getLineImage(event.message.id);
      var base64Image = Utilities.base64Encode(imageBlob.getBytes());
      var nutritionData = analyzeFoodWithGemini(base64Image, imageBlob.getContentType());
      if (nutritionData) {
        writeToUserSheet(nutritionData, userName, -1);
        replyMessage(replyToken, "□ Gemini 2.5 已成功為您記錄！
        " + formatNutritionText(nutritionData));
      }
    }
  } catch (error) {

```

```

    if (replyToken) replyMessage(replyToken, "□□ 系統診斷報告：
" + error.toString());
}
return ContentService.createTextOutput("OK");
}

// =====
// 3. 高階 AI 修正與統計報表核心邏輯
// =====
// 進階修正模組：智慧比對今日特定餐點並強制重置數值
function handleAdvancedCorrection(replyToken, userName, userMsg) {
    var ss = SpreadsheetApp.openById(SHEET_ID);
    var sheet = ss.getSheetByName(userName);
    if (!sheet || sheet.getLastRow() < 2) { replyMessage(replyToken, "尚未有任何歷史紀錄，無法執行修正。"); return; }

    var targetNum = 0;
    var numMatch = userMsg.match(/第\s*(\d+)\s*餐/);
    if (numMatch) targetNum = parseInt(numMatch[1]);

    var rows = sheet.getDataRange().getValues();
    var today = new Date(); today.setHours(0,0,0,0);
    var todayRowIndices = [];
    for (var i = 1; i < rows.length; i++) {
        var d = new Date(rows[i][0]); d.setHours(0,0,0,0);
        if (d.getTime() === today.getTime()) todayRowIndices.push(i + 1);
    }

    // 若未指定第幾餐，預設為今日最後一餐
    var targetRowIndex = (targetNum > 0 && targetNum <= todayRowIndices.length)
        ? todayRowIndices[targetNum - 1]
        : todayRowIndices[todayRowIndices.length - 1];

    if (!targetRowIndex) { replyMessage(replyToken, "找不到該序號的紀錄，請先傳送「今天統計」核對編號。"); return; }

    var oldText = sheet.getRange(targetRowIndex, 2).getValue();
    var prompt = "你是專業營養師。請修正以下餐點。原本內容：「" + oldText + "」。使用者提出的修正指令：「" + userMsg + "」。請重新估算每項食物正確熱量，以 JSON 格式回傳，嚴禁將數值設為 0。格式：{\"items\": [{\"name\": \"食物名\", \"calories\": 數值}], \"total_calories\": 總和, \"total_carbs\": 總和, \"total_protein\": 總和, \"total_fat\": 總和}";

    var apiUrl = "https://generativelanguage.googleapis.com/v1beta/models/" + GEMINI_MODEL + ":generateContent?key=" + GEMINI_API_KEY;
    var response = UrlFetchApp.fetch(apiUrl, {
        "method": "post", "contentType": "application/json", "payload": JSON.stringify({
            "contents": [{"parts": [{"text": prompt}]}],
            "generationConfig": {"responseMimeType": "application/json"}
        })
    });

    var rawText = JSON.parse(response.getContentText()).candidates[0].content.parts[0].text;
    var correctedData = JSON.parse(rawText.replace(/\\njson/g, "").replace(/\\n/g, "").trim());

```

```

writeToUserSheet(correctedData, userName, targetRowIndex);
replyMessage(replyToken, "□□ 修正成功 (經由 Gemini 2.5 重新校正) !"
+ formatNutritionText(correctedData));
}

// 今日明細與熱量統計
function queryTodayNutrition(userName) {
var ss = SpreadsheetApp.openById(SHEET_ID);
var sheet = ss.getSheetByName(userName);
if (!sheet) return "□ 尚未找到您的飲食資料庫。";
var rows = sheet.getDataRange().getValues();
var today = new Date(); today.setHours(0,0,0,0);
var tCal = 0, count = 0, list = "";
for (var i = 1; i < rows.length; i++) {
var d = new Date(rows[i][0]); d.setHours(0,0,0,0);
if (d.getTime() === today.getTime()) {
count++;
list += "
□ 填報編號 [" + count + "] :
" + rows[i][1] + "
";
tCal += Number(rows[i][2] || 0);
}
}
return count === 0 ? "□ 您今天尚未上傳任何食物照片喔 !" : "□ " + userName + " 的今日飲食清單 :
" + list + "
□ 今日熱量累積總計 : " + tCal.toFixed(0) + " 大卡";
}

// 本月加總趨勢與天數均值計算
function queryMonthlyNutrition(userName) {
var ss = SpreadsheetApp.openById(SHEET_ID);
var sheet = ss.getSheetByName(userName);
if (!sheet) return "□ 尚未找到您的飲食資料庫。";
var rows = sheet.getDataRange().getValues();
var now = new Date();
var tCal = 0, tCarb = 0, tProt = 0, tFat = 0, count = 0, days = {};
for (var i = 1; i < rows.length; i++) {
var d = new Date(rows[i][0]);
if (d.getMonth() === now.getMonth() && d.getFullYear() === now.getFullYear()) {
count++;
tCal += Number(rows[i][2] || 0); tCarb += Number(rows[i][3] || 0);
tProt += Number(rows[i][4] || 0); tFat += Number(rows[i][5] || 0);
days[d.toString()] = true;
}
}
if (count === 0) return "□ 您本月份尚未有任何紀錄。";
var dCount = Object.keys(days).length;
return "□ " + userName + " 本月控糖減重報表

```

```

" +
    "□ 統計月份：" + (now.getMonth() + 1) + "月
" +
    "-----
" +
    "□ 總攝取熱量：" + tCal.toFixed(0) + " 大卡
" +
    "□ 總碳水化合物：" + tCarb.toFixed(0) + " g
" +
    "□ 總蛋白質：" + tProt.toFixed(0) + " g
" +
    "□ 總脂肪攝取：" + tFat.toFixed(0) + " g
" +
    "-----
" +
    "□ 週期平均數據：
" +
    "• 有紀錄的天數：" + dCount + " 天
" +
    "• 每日平均熱量：" + (tCal/dCount).toFixed(0) + " 大卡";
}

// =====
// 4. 底層基礎工具箱 (資料庫寫入、API 請求)
// =====

function writeToUserSheet(data, userName, rowIndex) {
    var ss = SpreadsheetApp.openById(SHEET_ID);
    var sheet = ss.getSheetByName(userName) || ss.insertSheet(userName);
    if (sheet.getLastRow() === 0) {
        sheet.appendRow(["時間戳記", "食物明細", "總熱量", "碳水", "蛋白", "脂肪"]);
        sheet.getRange("A1:F1").setFontWeight("bold");
    }

    var detail = (data.items && Array.isArray(data.items))
        ? data.items.map(function(i){return i.name + " (" + (i.calories || 0) + "大卡");}).join("
")
        : "無明細數據";

    var rowData = [
        new Date(),
        detail.toString(),
        Number(data.total_calories || 0),
        Number(data.total_carbs || 0),
        Number(data.total_protein || 0),
        Number(data.total_fat || 0)
    ];

    if (rowIndex > 1) {
        sheet.getRange(rowIndex, 1, 1, 6).setValues([rowData]);
    } else {
        sheet.appendRow(rowData);
    }
}

```

```

    }
}

function analyzeFoodWithGemini(base64Image, mimeType) {
    var prompt = "你是一位頂尖的營養師。請用視覺精準分析照片。獨立拆解菜色（必須使用純繁體中文），並根據份量合理估計熱量（嚴禁回傳 0 或是空值）。請絕對以完整 JSON 回傳：{\n\"items\": [{\"name\": \"食物名\", \"calories\": 數值}], \"total_calories\": 總和, \"total_carbs\": 總和, \"total_protein\": 總和, \"total_fat\": 總和}";
    var apiUrl = "https://generativelanguage.googleapis.com/v1beta/models/" + GEMINI_MODEL + ":generateContent?key=" + GEMINI_API_KEY;

    var response = UrlFetchApp.fetch(apiUrl, {
        "method": "post", "contentType": "application/json", "payload": JSON.stringify({
            "contents": [{"parts": [{"text": prompt, "inline_data": {"mime_type": mimeType, "data": base64Image}}]}],
            "generationConfig": {"responseMimeType": "application/json"}
        })
    });
    var rawText = JSON.parse(response.getContentText()).candidates[0].content.parts[0].text;
    return JSON.parse(rawText.replace(/\\n/g, "").replace(/\\n/g, "").trim());
}

function formatNutritionText(data) {
    var itemsText = (data.items && Array.isArray(data.items))
        ? data.items.map(function(i){return "□ " + i.name + " : " + (i.calories || 0) + " 大卡"}).join(
            "\n")
        : "無細節";
    return "-----\n" + itemsText + "\n-----\n";
    "總計熱量：" + (data.total_calories || 0) + " 大卡\n";
    "碳水：" + (data.total_carbs || 0) + "g | 蛋白：" + (data.total_protein || 0) + "g | 脂肪：" + (data.total_fat || 0) + "g";
}

function getUserProfile(userId) {
    try {
        var response = UrlFetchApp.fetch("https://api.line.me/v2/bot/profile/" + userId, {"headers": {"Authorization": "Bearer " + LINE_ACCESS_TOKEN}});
        return JSON.parse(response.getContentText()).displayName;
    } catch (e) { return "外部使用者"; }
}

function getLineImage(messageId) {
    return UrlFetchApp.fetch('https://api-data.line.me/v2/bot/message/' + messageId + '/content', {"headers": {"Authorization": "Bearer " + LINE_ACCESS_TOKEN}}).getBlob();
}

function replyMessage(replyToken, text) {
    UrlFetchApp.fetch('https://api.line.me/v2/bot/message/reply', {
        "method": "post", "headers": {"Authorization": "Bearer " + LINE_ACCESS_TOKEN}, "contentType": "application/json",

```

```
"payload": JSON.stringify({"replyToken": replyToken, "messages": [{"type": "text", "text": text}]}))
});
}
```

3. 部署 Web 應用程式 (Webapp URL)

點擊 GAS 頁面右上角的「部署」->「新增部署作業」。

點擊左側小齒輪，選擇「Web 應用程式 (Web App)」。

設定配置：

說明：輸入 Gemini2.5_v1.0。

執行身分：選擇「我 (Your Account)」。

誰有權限存取：必須選擇「所有人 (Anyone)」。

點擊「部署」，並在隨後跳出的 Google 授權視窗中，點選「進階」->「前往應用程式 (不安全)」核准執行權限。

部署完成後，複製系統提供的「網頁應用程式 URL」。

第三章：串接驗證與雙向測試

將 GAS 與 LINE 後台連動，完成最後的閉環設定。

1. 在 LINE Developers 綁定 Webhook

回到 LINE Developers Console 的 Messaging API 頁籤。

找到「Webhook URL」，點擊「Edit」。

貼上剛剛在 GAS 複製的「網頁應用程式 URL」，點擊「Save」。

關鍵步驟：將「Use webhook」欄位切換為開啟狀態 (Enabled)。

點擊 Webhook URL 下方的「Verify」，若跳出 Success 即代表雙向串接完全成功。

重點整理

安全權限：每次修改 GAS 程式碼後，必須點擊「部署 -> 管理部署作業 -> 編輯 -> 選擇新版本」，最後點擊更新，修改才會在 LINE 機器人上生效。

零熱量防呆：本手冊在 Prompt 中實施了「視覺鏈加固」，強制要求 Gemini 2.5 進行重量與份量比對，全面防範 AI 將滷雞腿、排骨飯等認成 0 大卡的問題。

精準修復：若系統出現辨識瑕疵，在 LINE 發送「修正第 1 餐：將炸地瓜改為生菜沙拉」，試算表與 AI 就會同步覆蓋並更新今日報表。

TODO 檢查清單

[] 獲取並填寫 LINE Developers Channel Access Token 到程式碼第 4 行。

[] 註冊 Google AI Studio 獲取 Gemini API 金鑰並填寫到程式碼第 5 行。

[] 建立試算表並將其 ID 填寫到程式碼第 6 行。

[] 完成 GAS 部署並將 Webhook 網址對接回 LINE 後台點擊 Verify 驗證。